

ACTICAL_WEEK-04

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
Last login: Mon Aug 17 08:22:14 on console
root@Vishnu:~# cd ~/OSSP
root@Vishnu:~# pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory for Week 4.

```
root@Vishnu:~# nano src/wait_waitpid.c
cat src/wait_waitpid.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2, child3;
    int status;
    pid_t finished;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    child1 = fork();
    if (child1 == 0)
    {
        printf("Child 1 started, PID = %d, Parent PID = %d\n",
              getpid(), getpid());
        sleep(2);
        exit(10);
    }

    child2 = fork();
    if (child2 == 0)
    {
        printf("Child 2 started, PID = %d, Parent PID = %d\n",
              getpid(), getpid());
        sleep(1);
        exit(11);
    }

    child3 = fork();
    if (child3 == 0)
    {
        printf("Child 3 started, PID = %d, Parent PID = %d\n",
              getpid(), getpid());
        sleep(6);
        exit(12);
    }

    printf("All 3 child processes have been created.\n");
    printf("Waiting wait():\n");
    finished = wait(&status);
    if (WIFEXITED(status))
    {
        printf("wait() detected child PID %d finished with exit status %d\n",
              finished, WEXITSTATUS(status));
    }

    printf("Waiting waitpid():\n");
```

Figure 2(a): Child process creation and `wait()` implementation

```
printf("\nUsing waitpid():\n");

pid_t children[] = {child1, child2, child3};

for (int i = 0; i < 3; i++)
{
    finished = waitpid(children[i], &status, 0);

    if (finished > 0 && WIFEXITED(status))
    {
        printf("waitpid() detected child PID %d finished with exit status %d\n",
              finished, WEXITSTATUS(status));
    }
}

printf("\nParent process completed.\n");

return 0;
}
root@Vishnu:~#
```

Figure 2(b): `waitpid()` synchronization and parent process completion

```

root@Vishnu:~# gcc src/wait_waitpid.c -o bin/wait_waitpid
root@Vishnu:~# ./bin/wait_waitpid
Parent Process Started
Parent PID: 20409
Child 1 started. PID = 20412, Parent PID = 20409
Child 2 started. PID = 20413, Parent PID = 20409
All 3 child processes have been created.

Using wait():
Child 3 started. PID = 20414, Parent PID = 20409
wait() detected child PID 20412 finished with exit status 10

Using waitpid():
waitpid() detected child PID 20413 finished with exit status 11
waitpid() detected child PID 20414 finished with exit status 12

Parent process completed.
root@Vishnu:~#

```

Figure 3: Successful compilation of the wait() and waitpid() program and execution of multiple child processes using wait() and waitpid()

```

root@Vishnu:~# git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/parser.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bin/escape_parser
        bin/parser
        bin/quote_parser
        bin/wait_waitpid
        double_quote_flowchart.txt
        escape_flowchart.txt
        parser_flowchart.txt
        process_flowchart.txt
        quote_flowchart.txt
        src/escape_parser.c
        src/parser.c.save
        src/quote_parser.c
        src/wait_waitpid.c
        wait_flowchart.txt

no changes added to commit (use "git add" and/or "git commit -a")
root@Vishnu:~#

```

Figure 4: Git status after implementing the wait() and waitpid() program

TASK 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
Last login: Mon Aug 17 10:29:44 on ttys001
root@Vishnu:~# ~ % cd ~/OSSP
root@Vishnu:~# OSSP % cd ~/OSSP
root@Vishnu:~# OSSP % pwd
/Users/Vishnu/OSSP
root@Vishnu:~# OSSP %
```

Figure 1: OSSP project directory for Week 4.

```
root@Vishnu:~# nano src/zombie_process.c
root@Vishnu:~# cat src/zombie_process.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");

        sleep(20);

        printf("Parent terminating...\n");
    }

    return 0;
}
root@Vishnu:~#
```

Figure 2: C program for creating a zombie process

```

root@Vishnu:~# gcc src/zombie_process.c -o bin/zombie_process
root@Vishnu:~# ./bin/zombie_process
Parent process: PID = 23503
Child PID = 23512
Parent sleeping...
Child process: PID = 23512
Child process terminating...
Parent terminating...
root@Vishnu:~#

```

Figure 3: Successful compilation of the zombie process program and execution of the zombie process program with parent and child process termination.

```

root@Vishnu:~# ./bin/zombie_process
Parent process: PID = 25522
Child PID = 25525
Parent sleeping...
Child process: PID = 25525
Child process terminating...
Parent terminating...
root@Vishnu:~#

```

Figure 4: Execution of the program demonstrating parent and child process termination

```

root@Vishnu:~# ps -p 25525 -o pid,ppid,state,command
  PID  PPID  STAT  COMMAND
 25525 25522  Z+    <defunct>
root@Vishnu:~#

```

Figure 5: Identification of the zombie child process using the process table

```

root@Vishnu:~# cat src/zombie_process.c
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent waiting for child...\n");
        wait(NULL);
        printf("Child process has been reaped.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}

```

```

root@Vishnu:~# ./bin/zombie_process
Parent waiting for child...
Child process: PID = 31287
Child process terminating...
Child process has been reaped.
Parent terminating...
root@Vishnu:~#

```

Figure 6: Successful execution of the modified program with wait(